

A Recurrent Neural Network (RNN) is a type of neural network designed to work with sequential data—data where order matters.

Unlike regular neural networks, RNNs have memory.

They process data step-by-step and remember previous information using loops.

Example:

- Sentence: *"I love learning AI"*
- The meaning of each word depends on the previous words
- RNN keeps track of earlier words while processing later ones

At each step:

- Input = current data (like a word)
- Hidden state = memory from previous step
- Output = result for that step

Current Output = $f(\text{Current Input} + \text{Previous Memory})$

RNN Structure

An RNN looks like a chain:

$\text{Input}_1 \rightarrow \text{Hidden}_1 \rightarrow \text{Hidden}_2 \rightarrow \text{Hidden}_3 \rightarrow \text{Output}$

Each hidden layer passes information to the next one

Applications:

- Language translation (Google Translate)
- Speech recognition
- Text generation
- Time-series prediction (stock prices, weather)
- Chatbots

Limitations:

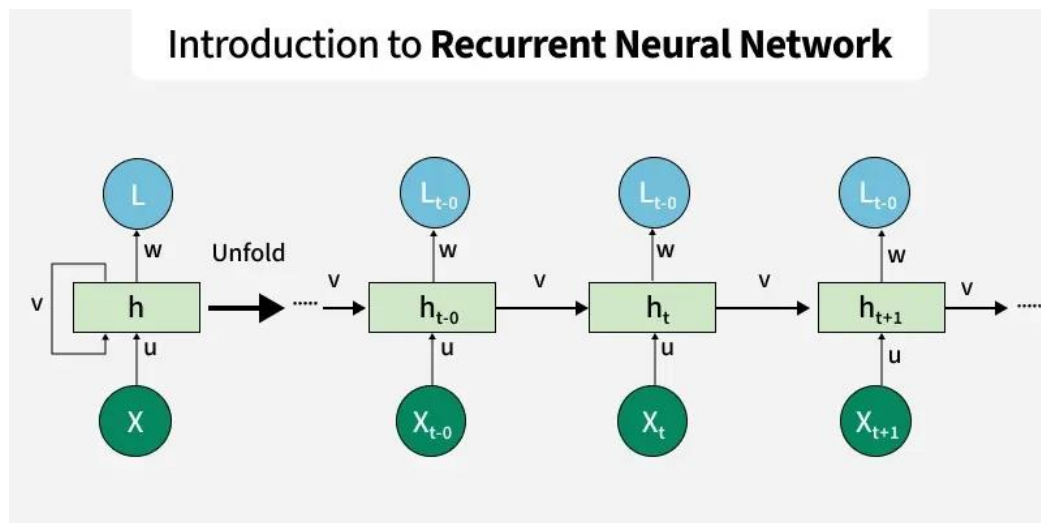
- Vanishing gradient problem (hard to learn long sequences)
- Forget long-term information

Example:

Input:

"I am going to the ____"

RNN uses previous words and predicts:
“market” or “school”



Recurrent Neural Networks (RNNs) and Variants:

Sequential Data:

Sequential data is any data where the order of elements matters.

- If you change the order, the meaning changes.

Examples:

- Text: “I love AI” $\neq$ “AI love I”
- Speech signals (audio over time)
- DNA sequences
- Music notes

Key Characteristics:

- Has a specific order
- Each element depends on previous elements
- Can be short-term or long-term dependent

Types of Sequential Data

1. One-to-One

- Input $\rightarrow$ Output (normal neural network)

2. One-to-Many

- One input → sequence output
Example: image captioning

Input: one image

Output: a sequence of words forming a sentence

Example:

Input image: a picture of a dog playing with a ball in a park

Output caption:

“A dog is playing with a ball in the park.”

3. Many-to-One

- Sequence input → single output
Example: sentiment analysis

Input sentence:

“The movie was absolutely fantastic and exciting”

Output: Positive

Input sentence:

“The product is very bad and not worth the money”

Output: Negative

4. Many-to-Many

- Sequence input → sequence output
Example: translation

Machine Translation means:

Input: sentence in one language

Output: sentence in another language

Example:

Input (English):

“I love deep learning”

Output (French):

“J’aime l’apprentissage profond”

Time Series Data:

Time series data is a special type of sequential data where:
Data is collected at specific time intervals

Examples:

- Stock prices (daily)
- Temperature (hourly)
- Heart rate (per second)
- Sales data (monthly)

Properties of Time Series

1. Temporal Dependency

- Past values influence future values

2. Stationarity

- Data properties remain constant over time
Important for modeling

3. Autocorrelation

- Relationship between values at different time lags

Components of Time Series

Time series data usually has these patterns:

1. Trend

- Long-term increase or decrease
Example: rising population

2. Seasonality

- Repeating pattern at fixed intervals
Example: higher ice cream sales in summer

3. Cyclic Pattern

- Irregular long-term fluctuations
Example: economic cycles

4. Noise

Random variation (unpredictable part)

Normal neural networks are not enough because

A standard feedforward neural network assumes every input is independent. It does not remember previous inputs.

For example, in a sentence:

- “The movie was not ... good”
To understand the word “good,” the model should remember “not.”

In time series:

- Today’s stock price is related to yesterday’s and earlier prices.

This need for memory led to the development of Recurrent Neural Networks.

A Recurrent Neural Network is a neural network designed for sequential data. It processes one element at a time and carries forward information from previous steps using a hidden state.

It has a kind of memory.

At each time step:

- It takes the current input
- It combines it with the previous hidden state
- It produces a new hidden state
- It may also produce an output

So the network remembers something from the past.

Basic Structure of an RNN

Suppose we have a sequence:

$$x_1, x_2, x_3, \dots, x_T$$

At each time step t :

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Where:

- x_t :input at time step t
- h_t :hidden state at time step t
- y_t :output at time step t
- W_{xh} :input-to-hidden weight matrix

- W_{hh} :hidden-to-hidden recurrent weight matrix
- W_{hy} :hidden-to-output weight matrix
- b_h, b_y :bias terms
- ϕ : activation function such as tanh or ReLU

The same weights are used at every time step.

That means:

- the network applies the same rule again and again
- the model can process sequences of different lengths

Sentence: "I love deep learning"

Step 1

Input: "I"

- Convert "I" into a vector
- Combine with previous hidden state
- Since it is the first word, hidden state may start as zeros
- Produce new hidden state h_1

Step 2

Input: "love"

- Combine current word vector with h_1
- Produce h_2

Step 3

Input: "deep"

Combine current word vector with h_2

Produce h_3

Step 4

Input: "learning"

Combine current word vector with h_3

Produce h_4

So the hidden state acts like a summary of all previous words.

RNN Basics

A Recurrent Neural Network (RNN) is a neural network designed for sequential data, where:

- Output depends on current input + previous inputs
- It has a memory (hidden state)

Structure of RNN

At each time step t :

- Input $\rightarrow x_t$
- Hidden state $\rightarrow h_t$
- Output $\rightarrow y_t$

The hidden state is updated as:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

Output is:

$$y_t = g(W_y h_t)$$

- Same weights are used at every time step
- Information flows through time
- Hidden state acts like memory

$x_1 \rightarrow h_1 \rightarrow y_1$

↓

$x_2 \rightarrow h_2 \rightarrow y_2$

↓

$x_3 \rightarrow h_3 \rightarrow y_3$

Back Propagation Through Time:

BPTT is the training algorithm used for RNNs.

It is basically normal backpropagation applied to the unrolled RNN across time steps.

When you unroll the RNN for T time steps, it becomes like a deep network with T layers, one layer per time step.

Then, gradients are computed from the last time step backwards to the first

Forward Pass in BPTT

Suppose sequence length is 3.

Inputs:

$$x_1, x_2, x_3$$

Hidden states:

$$h_1, h_2, h_3$$

Outputs:

$$y_1, y_2, y_3$$

Loss:

$$L = L_1 + L_2 + L_3$$

At each step:

1. Compute hidden state
2. Compute output
3. Compute loss at that step

Total loss is the sum across time.

Backward Pass in BPTT

Now we calculate gradients of the loss with respect to weights.

Because hidden states depend on previous hidden states, the gradient at one time step affects earlier time steps too.

For example:

- h_3 depends on h_2
- h_2 depends on h_1

So error from later time steps flows backward through many earlier steps.

That is why it is called backpropagation through time.

Backpropagation Through Time (BPTT)

BPTT is the training algorithm used for RNNs.

It is an extension of backpropagation, applied over time steps

Steps in BPTT

1. Forward Pass

- Input sequence is processed step-by-step
- Outputs and hidden states are stored

2. Compute Loss

- Calculate total error across all time steps:

$$L = \sum_{t=1}^T L_t$$

3. Backward Pass (Through Time)

- Gradients are propagated backwards from last time step to first
- Uses chain rule
- Each weight affects multiple time steps

Gradient Flow

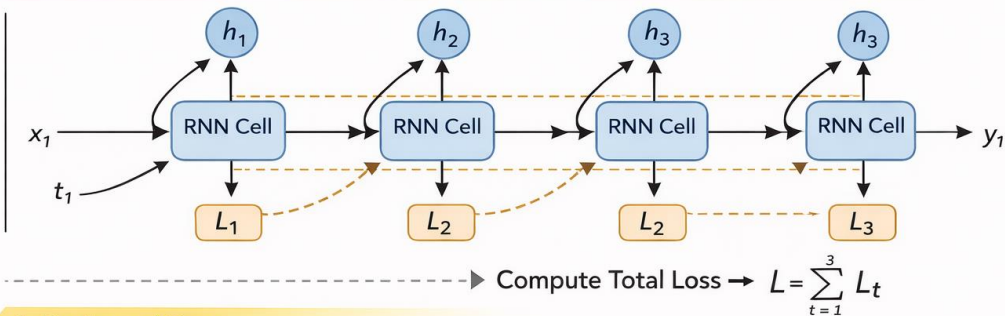
The gradient at time step t depends on:

$$\frac{\partial L}{\partial W} = \sum_t \frac{\partial L_t}{\partial W}$$

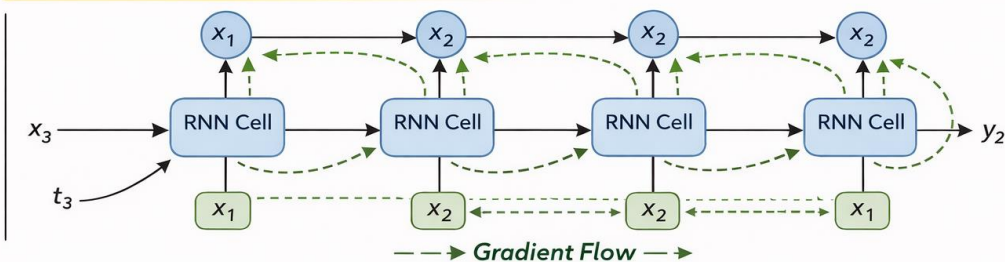
- Earlier steps receive gradients through many layers
- Leads to vanishing/exploding gradients

Backpropagation Through Time (BPTT)

1. Forward Pass



2. Backward Pass



Vanishing gradient means the gradient becomes extremely small as it travels backward through many time steps.

Then early time steps learn very little.

Why does it happen?

When we repeatedly multiply numbers smaller than 1, the result becomes tiny.

Example:

$$0.5 \times 0.5 \times 0.5 \times 0.5 = 0.0625$$

If we continue:

$$0.5^{10} = 0.000976$$

Very small.

In RNNs, if derivatives and recurrent weights are less than 1 in magnitude, repeated multiplication makes gradients almost zero.

Effect of Vanishing Gradient

The model cannot learn long-term dependencies.

Example in language

Sentence:

“The flowers in the garden near the river are beautiful.”

To decide that “are” matches “flowers,” the model should remember “flowers” from many words earlier.

A simple RNN may forget it.

Example in time series

To predict a value today, maybe data from 30 days ago matters.

A simple RNN may fail to learn that.

So it learns only short-term patterns, not long-term ones.

Traditional RNNs often use:

- sigmoid
- tanh

These activations can worsen vanishing gradients because their derivatives are often small.

Sigmoid derivative

Maximum derivative is only 0.25

So repeated multiplication quickly shrinks gradients.

Tanh derivative

Better than sigmoid, but still can suffer from shrinking when values saturate.

Better initialization

Careful initialization of recurrent weights can help.

Example:

- orthogonal initialization
- Xavier or He initialization depending on activation

This reduces extreme growth or shrinking of gradients.

Better activation functions

ReLU can sometimes help in some recurrent architectures, though plain RNNs with ReLU may also be unstable.

Gated architectures

This is the most important practical solution.

Use:

- LSTM
- GRU

These were designed to preserve useful information over longer time ranges.

RNN Variants

1. Vanilla RNN

This is the basic RNN.

Advantages

- simple
- fewer parameters
- easy to understand

Disadvantages

- poor at long-term dependencies

suffers heavily from vanishing gradient

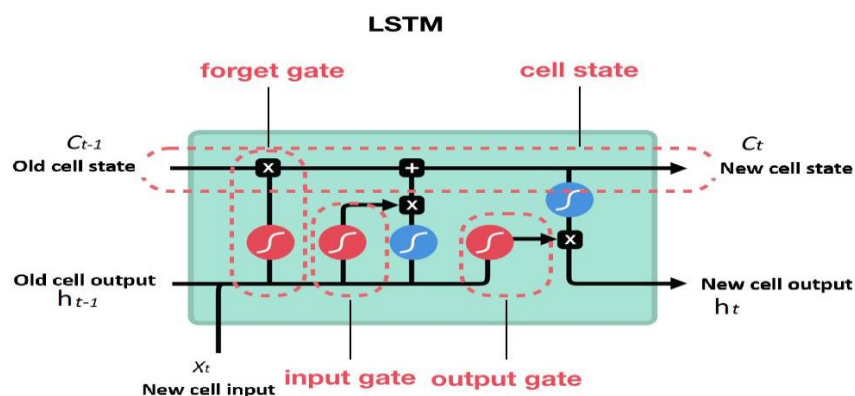
LSTM: Long Short-Term Memory

LSTM is a special type of RNN designed to handle long-term dependencies better.

It introduces a memory cell and gates to control information flow.

Instead of blindly updating memory, LSTM decides:

- what to forget
- what new information to add
- what to output



This makes memory control much better.

Components of LSTM

An LSTM has:

- cell state C_t
- hidden state h_t
- forget gate
- input gate
- candidate memory
- output gate

1. Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t])$$

2. Input Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t])$$

3. Candidate Memory

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t])$$

4. Cell State Update

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

5. Output Gate

$$o_t = \sigma(W_o[h_{t-1}, x_t])$$

Final Output

$$h_t = o_t \cdot \tanh(C_t)$$

Advantages

- Handles long-term dependencies
- Stable gradients

GRU: Gated Recurrent Unit

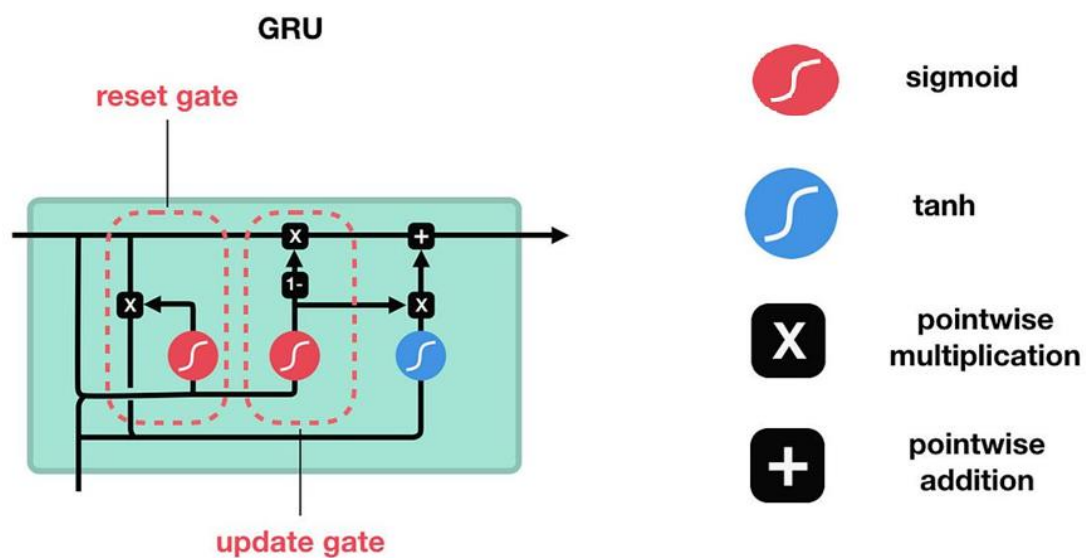
GRU is a simpler version of LSTM.

It has:

- update gate
- reset gate

It does not have a separate cell state like LSTM.

GRU combines memory and hidden state in a simpler way.



Gates

1. Update Gate

$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

2. Reset Gate

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

3. Candidate Hidden State

$$\tilde{h}_t = \tanh(W[h_t \cdot h_{t-1}, x_t])$$

Final Output

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$$

Applications (Text, Speech, Music):

Text Processing

- Language modeling
- Sentiment analysis
- Machine translation

◆ Speech Processing

- Speech recognition
- Speaker identification

◆ Music Generation

- Melody prediction
- Sequence generation

Sequence-to-Sequence Models:

Concept

Maps **input sequence** → **output sequence**

$$X = (x_1, \dots, x_n) \rightarrow Y = (y_1, \dots, y_m)$$

Architecture

Encoder

- Processes input sequence
- Outputs context vector

Decoder

- Generates output sequence

Problem

Single context vector → bottleneck

Attention Mechanism

- Focus on relevant parts

- Improves performance

Applications

- Machine translation
- Chatbots
- Text summarization

Speech Recognition

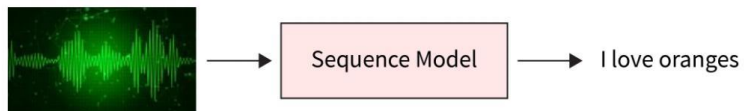
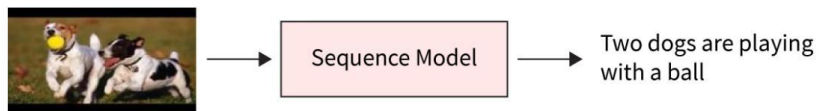


Image Captioning



Subtitle Generator

